# Pensamento Computacional
## INF05008

### Aula 14 – Dados Condicionais

Prof. Lucas Nunes Alegre

Universidade Federal do Rio Grande do Sul

Instituto de Informática

Departamento de Informática Teórica

Slides baseados no material da Profª. Leila Ribeiro.

# Definição de Estrutura

```
data NomeEstrutura:
    # Um elemento do conjunto NomeEstrutura tem o formato:
    nome-estrutura(
        atr1 :: Tipo1,      # <Interpretação do atr1>
        ...
        atrn :: TipoN       # <Interpretação do atrn>
    )
end
```

# Tipos de Dados

Dados podem ser de vários tipos (conjuntos):

- Number
- Boolean
- String
- Image
- Estruturas (Ponto, Aluno, Musico, Carro, ...)
- ...

# Tipos de Dados

Muitas vezes, precisamos representar dados que possuem mais de uma variante:

- O dado pode estar em uma entre várias formas possíveis (OU)
- Cada forma pode ter campos diferentes
- Precisamos saber qual variante está sendo usada
- Cada variante pode exigir tratamento diferente no programa

Exemplo: Um pagamento pode ser:

- Dinheiro (valor)
- Cartão (número, validade)
- Pix (chave)
- Um pagamento é Dinheiro OU Cartão OU Pix — nunca todos ao mesmo tempo.

# Definição do Conjunto **Ponto**

Na aula anterior, definimos um dado estruturado Ponto:

```
data Ponto:
    # Um elemento do conjunto Ponto tem o formato:
    ponto2d(
        coord-x :: Number,  # Coordenada x do ponto
        coord-y :: Number   # Coordenada y do ponto
    )
end
```

P. Mas e se quisermos trabalhar com pontos no espaço 3D também?

- Podemos adicionar uma nova variante ao tipo Ponto!

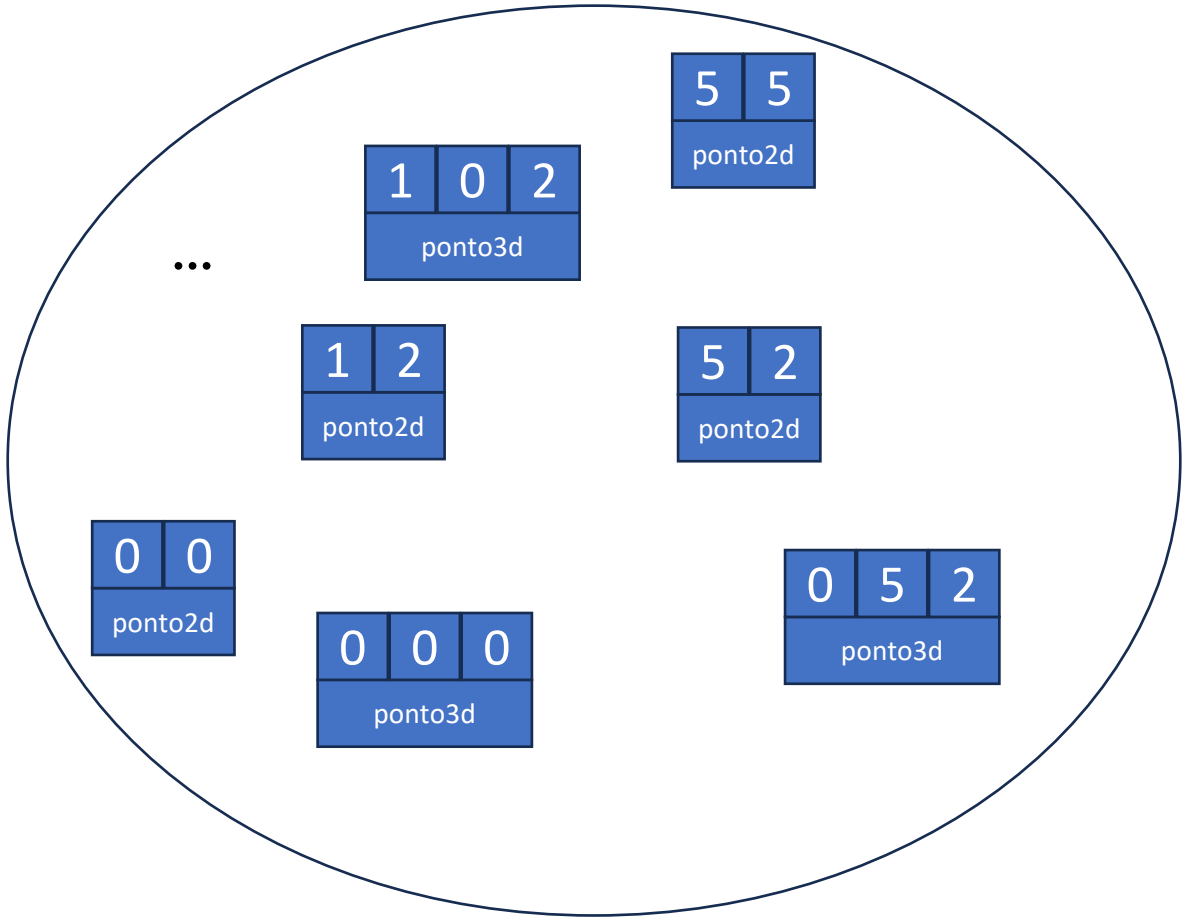
# Definição do Conjunto **Ponto**

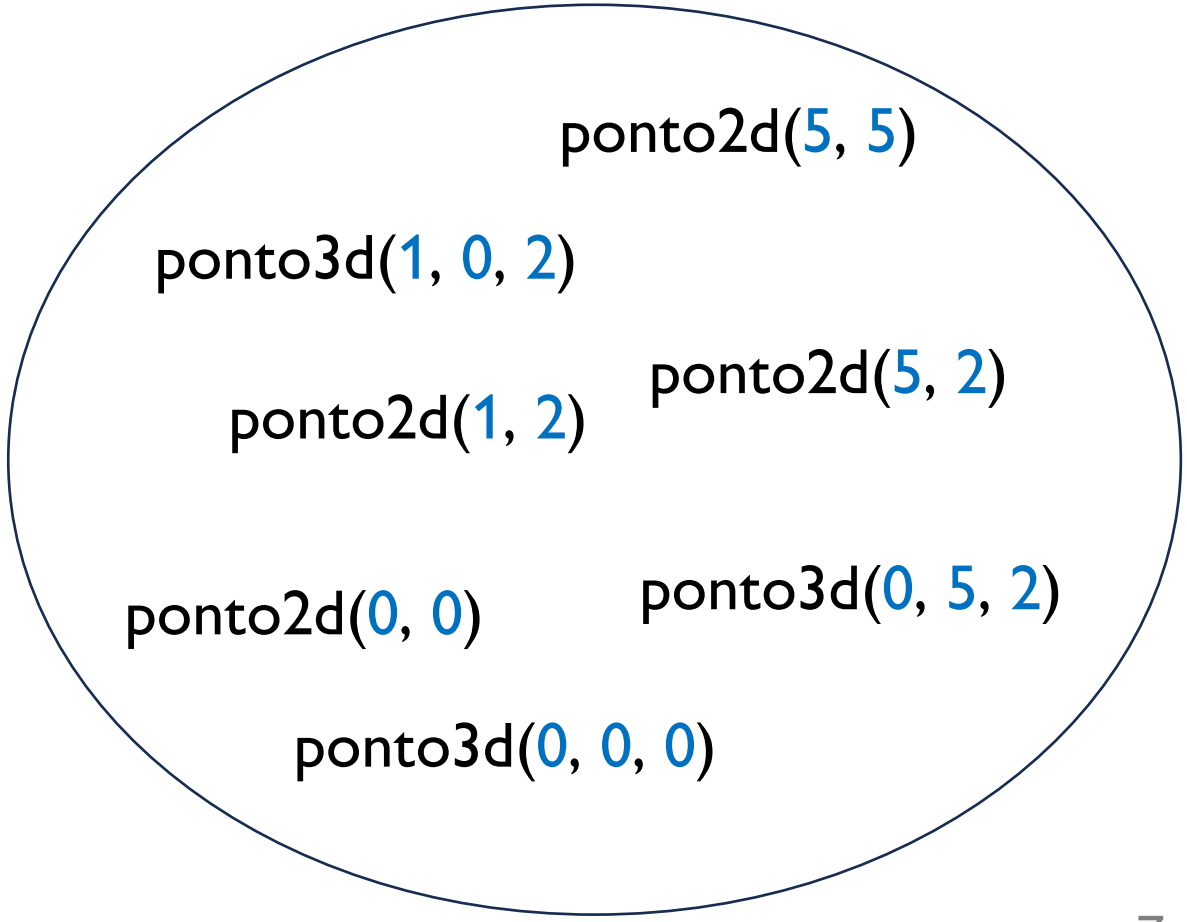
```
data Ponto:
  # Um elemento do conjunto Ponto tem o formato:
  | ponto2d(
      coord-x :: Number,    # Coordenada x do ponto
      coord-y :: Number     # Coordenada y do ponto
    )
  # Ou:
  | ponto3d(
      coord-x :: Number,    # Coordenada x do ponto
      coord-y :: Number,    # Coordenada y do ponto
      coord-z :: Number     # Coordenada z do ponto
    )
end
```

- Cada "|" introduz uma nova variante.
- O tipo Ponto é a UNIÃO de ponto2d e ponto3d.

# Conjunto **Ponto**

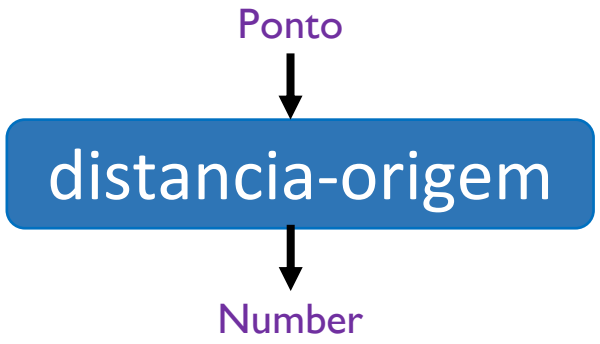
- O tipo Ponto é a UNIÃO de ponto2d e ponto3d.



# distancia-origem

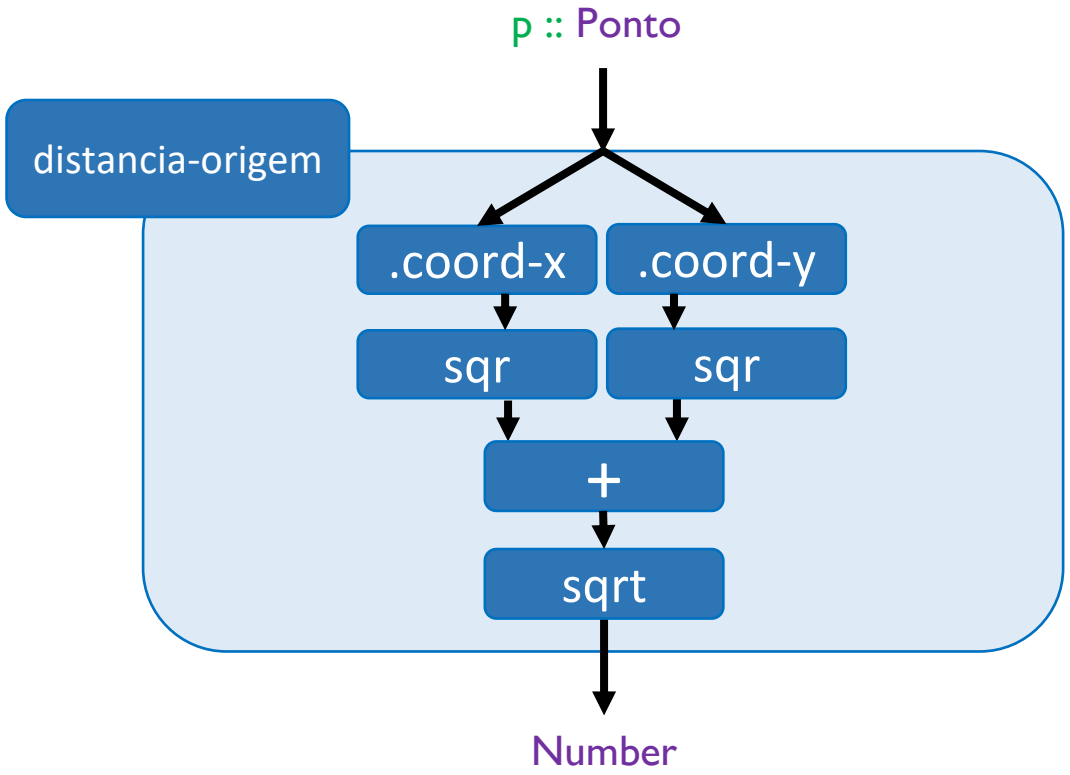
- Dado um ponto no plano, retorna a distância deste ponto à origem.
- Exemplos:
  - distancia-origem( ponto2d(2, 0) ) is 2
  - distancia-origem( ponto2d(3, 2) ) is 3.6




$$\text{distancia-origem}(\text{ponto}) = \sqrt{(\text{coordenada x do ponto})^2 + (\text{coordenada y do ponto})^2}$$

# distancia-origem

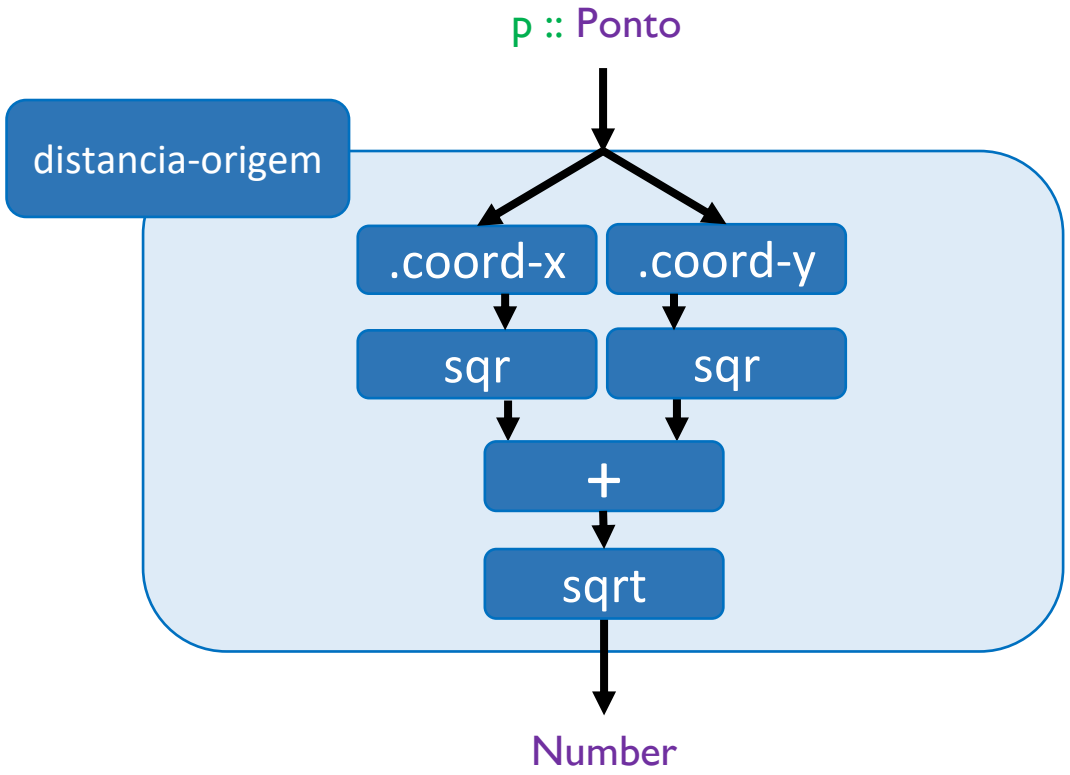
$$\text{distancia-origem}(\text{ponto}) = \sqrt{(\text{coordenada x do ponto})^2 + (\text{coordenada y do ponto})^2}$$




```
fun distancia-origem(p :: Ponto) -> Number:
    doc: "Calcula a distância do ponto p até a
origem (0,0)."
    sqrt(sqr(p.coord-x) + sqr(p.coord-y))
where:
    distancia-origem(ponto2d(2,0)) is 2
    distancia-origem(ponto2d(3,4)) is 5
end
```

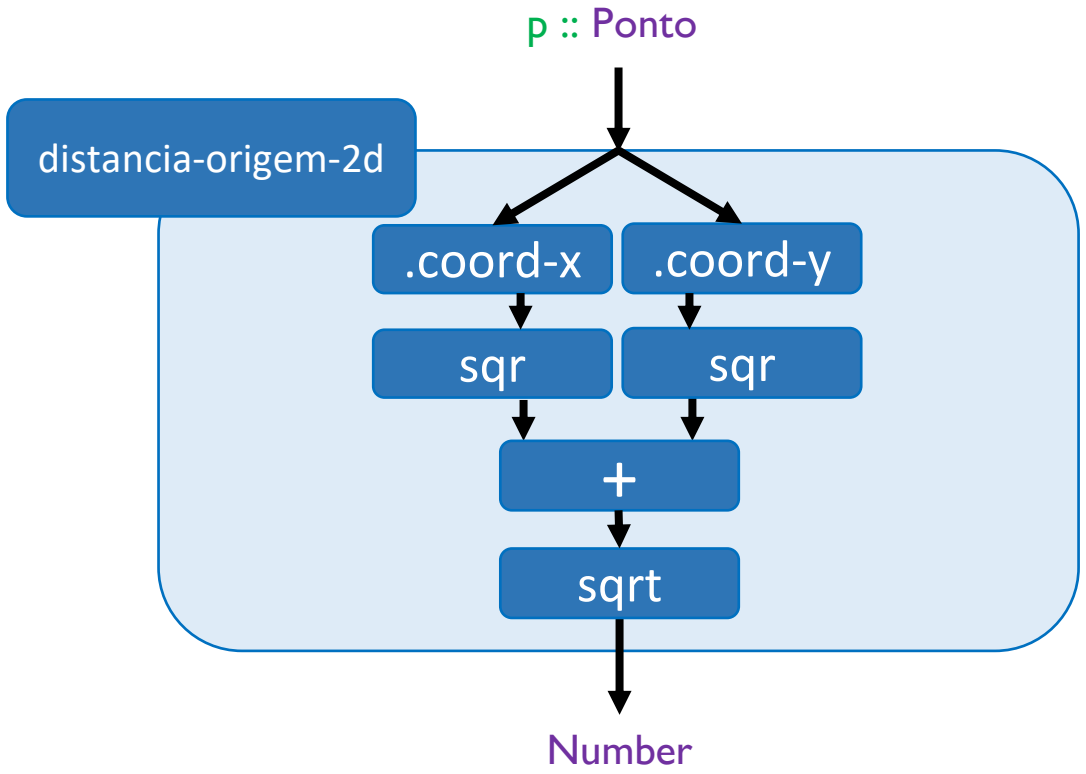
# distancia-origem

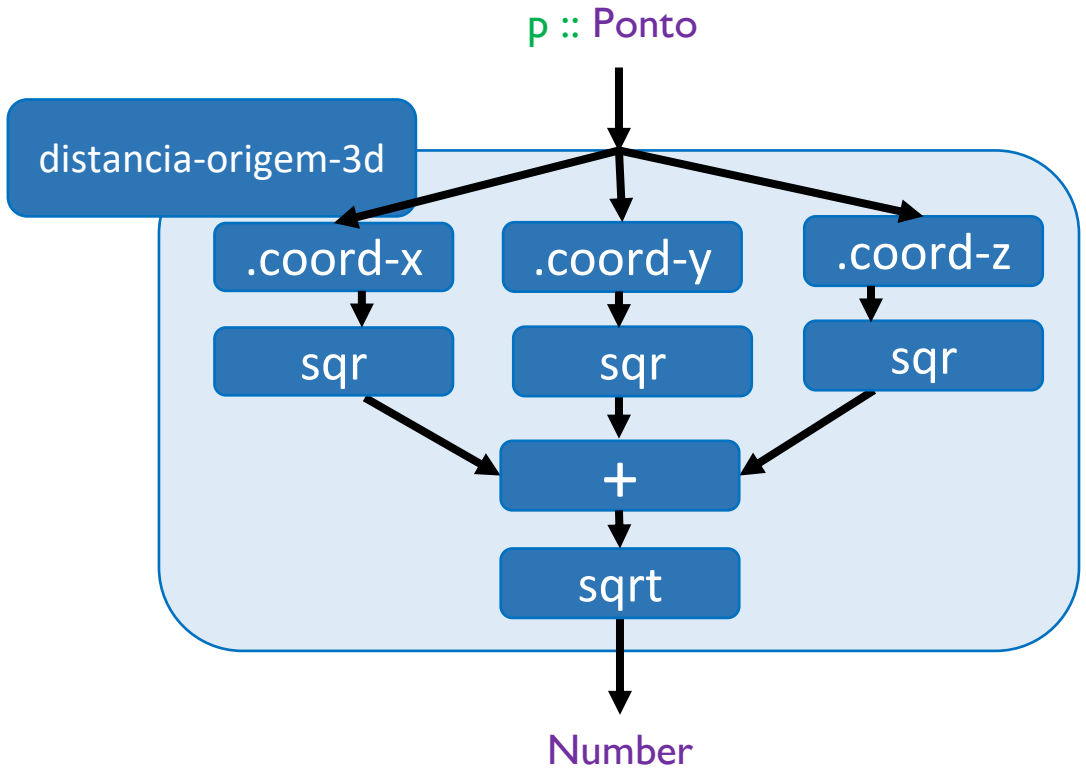
Como adaptamos essa função para a nova definição do tipo Ponto que include pontos 3D?




```
fun distancia-origem(p :: Ponto) -> Number:
    doc: "Calcula a distância do ponto p até a
origem (0,0)."
    sqrt(sqr(p.coord-x) + sqr(p.coord-y))
where:
    distancia-origem(ponto2d(2,0)) is 2
    distancia-origem(ponto2d(3,4)) is 5
end
```

# distancia-origem




$$\text{distancia-origem-2d}(\text{ponto}) = \sqrt{(\text{coord x do ponto})^2 + (\text{coord y do ponto})^2}$$

$$\text{distancia-origem-3d}(\text{ponto}) = \sqrt{(\text{coord x do ponto})^2 + (\text{coord y do ponto})^2 + (\text{coord z do ponto})^2}$$

# distancia-origem

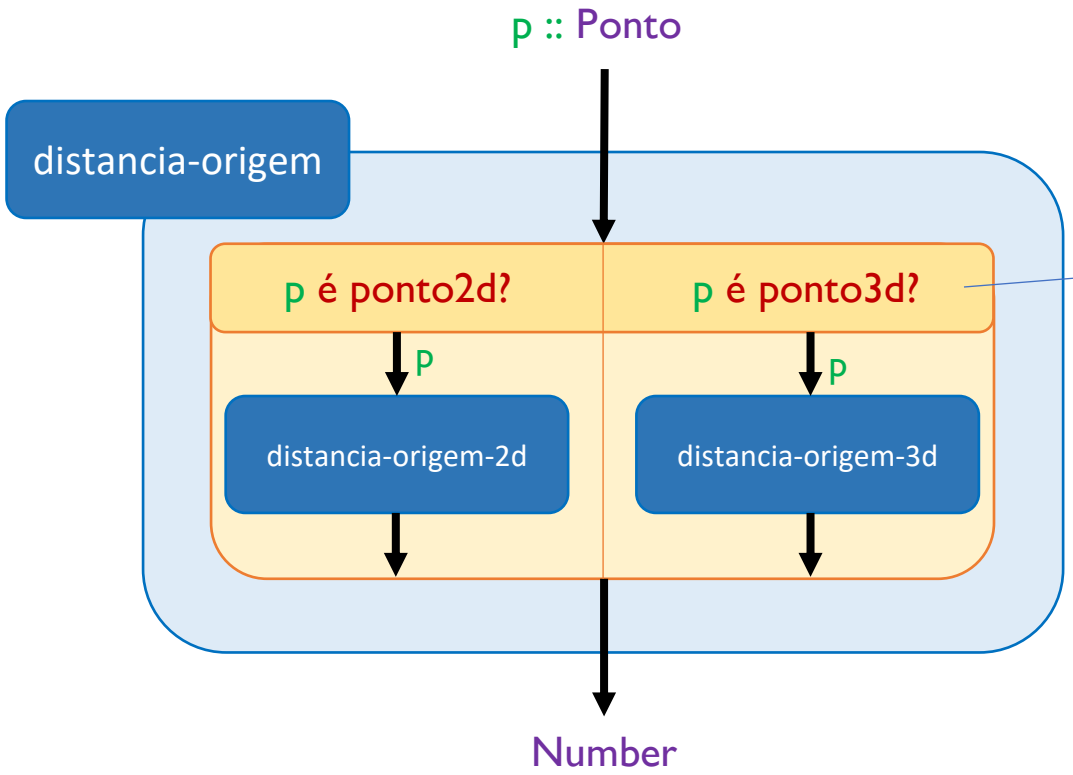
```
fun distancia-origem-2d(p :: Ponto) -> Number:
    doc: "Calcula a distância do ponto p até a origem.”
    sqrt(sqr(p.coord-x) + sqr(p.coord-y))
where:
    distancia-origem(ponto2d(2,0)) is 2
    distancia-origem(ponto2d(3,4)) is 5
end
```

```
fun distancia-origem-3d(p :: Ponto) -> Number:
  doc: "Calcula a distância do ponto p até a origem."
  sqrt(sqr(p.coord-x) + sqr(p.coord-y) + sqr(p.coord-z))
where:
  distancia-origem(ponto3d(1, 2, 2)) is 3
end
```

$$\text{distancia-origem-2d}(\text{ponto}) = \sqrt{(\text{coord x do ponto})^2 + (\text{coord y do ponto})^2}$$

$$\text{distancia-origem-3d}(\text{ponto}) = \sqrt{(\text{coord x do ponto})^2 + (\text{coord y do ponto})^2 + (\text{coord z do ponto})^2}$$

# distancia-origem




Como podemos checar qual a variante do ponto?

# Expressão **cases**

Em Pyret, distinguimos as variantes de um dado condicional com a expressão **cases**:

```
cases (Ponto) p:
   | ponto2d(x, y) => distancia-origem-2d(p)
   | ponto3d(x, y, z) => distancia-origem-3d(p)
end
```

- p é uma expressão que avalia para um valor do tipo Ponto
- x, y, z são nomes dados para os atributos de ponto2d/ponto3d
- O lado direito de todo "=>" é uma expressão

# Expressão **cases**

Em Pyret, distinguimos as variantes de um dado condicional com a expressão **cases**:

```
cases (Ponto) p:
   | ponto2d(x, y) => sqrt(sqr(x) + sqr(y))
   | ponto3d(x, y, z) => sqrt(sqr(x) + sqr(y) + sqr(z))
end
```

- p é uma expressão que avalia para um valor do tipo Ponto
- x, y, z são nomes dados para os atributos de ponto2d/ponto3d
- O lado direito de todo "=>" é uma expressão

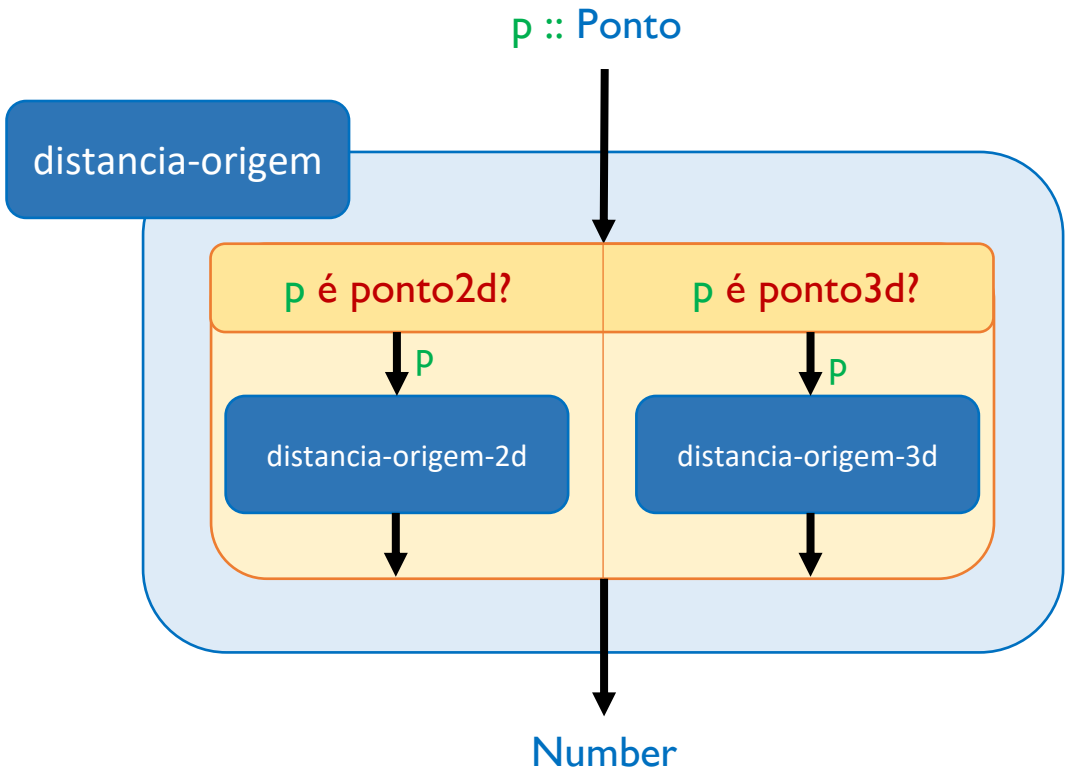
# Expressão **cases**

Em Pyret, distinguimos as variantes de um dado condicional com a expressão **cases**:

```
cases (Tipo) e:
   | variante-1(x, y) => <expressão>
   | ...
   | variante-n(x, y, z) => <expressão>
end
```

- e é uma expressão que avalia para um valor do tipo Tipo
- Primeiro, e é avaliado e é checado se e é do tipo Tipo. Se não for, erro de execução.
- Se é do Tipo, checa qual variante, e retorna a expressão a direita do => correspondente .

# distancia-origem




```
fun distancia-origem(p :: Ponto) -> Number:
  doc: "Calcula a distância do ponto p até a
origem."
  cases (Ponto) p:
    | ponto2d(x, y) => distancia-origem-2d(p)
    | ponto3d(x, y, z) => distancia-origem-3d(p)
  end
where:
  distancia-origem(ponto2d(2, 0)) is 2
  distancia-origem(ponto3d(1, 2, 2)) is 3
end
```

# Teste de Tipo

Pyret oferece funções para testar tipos:

- is-number :: Any → Boolean
- is-boolean :: Any → Boolean
- is-string :: Any → Boolean
- is-image :: Any → Boolean

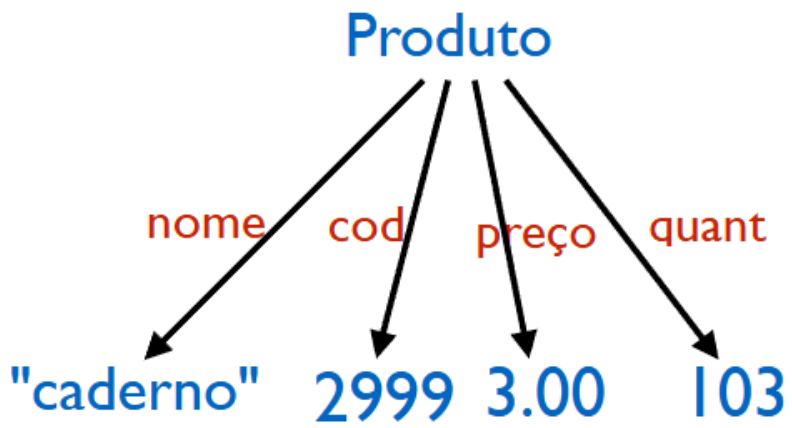
Para cada dado estruturado definido, também é criado automaticamente uma função:

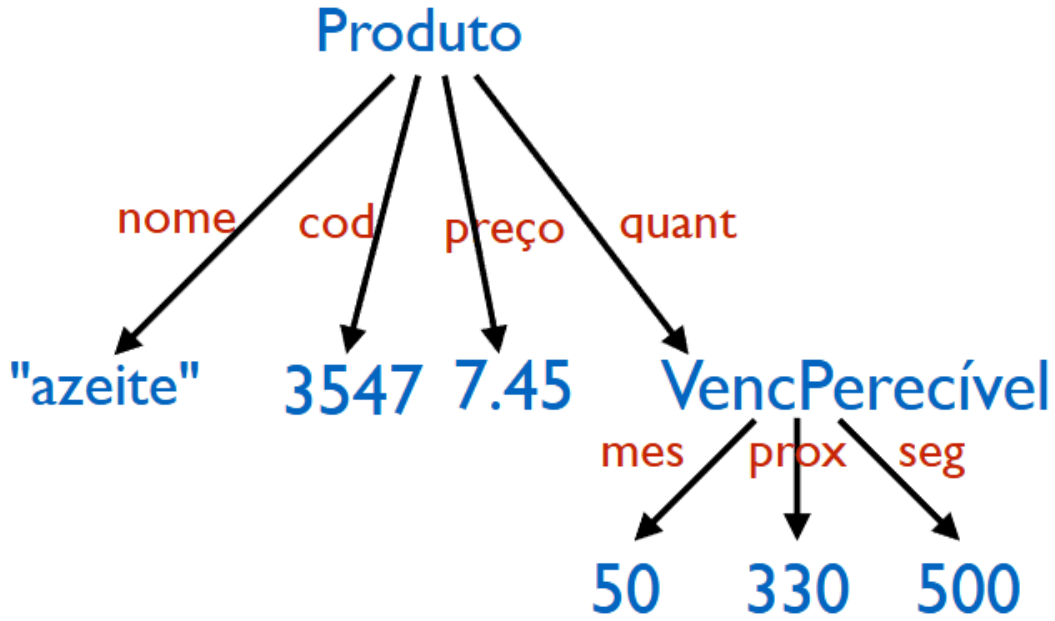
- is-ponto2d :: Any → Boolean
- is-ponto3d :: Any → Boolean
- is-musico :: Any → Boolean

"Any" significa qualquer tipo.

# Exemplo

Um supermercado tem produtos perecíveis e não-perecíveis. Todos os produtos tem um nome, código e preço. Para os produtos não perecíveis, registra-se a quantidade em estoque, enquanto que para os perecíveis, registram-se as quantidades que vencem no mês corrente, no próximo mês, e nos meses seguintes.



# Conjunto QuantidadeVencimento

```
data QuantidadeVencimento:
  | quant(n :: Number)
  # Onde n é a quantidade de itens não perecíveis
  | venc-perecivel(mes :: Number, prox :: Number, seg :: Number)
  # Onde mes, prox e seg são a quantidade de itens que vencem no
mês, próximo mês e meses seguintes.
end
```

# Conjunto Produto

```
data Produto:
    produto(
    nome :: String,                         # Nome do produto
    cod :: Number,                          # Código do produto
    preco :: Number,                        # Preço do produto
    quant :: QuantidadeVencimento           # Quantidade e vencimento do produto
  )
end
```

# Exercícios

1. Definir constantes com os elementos dos conjuntos QuantidadeVencimento e Produto.
2. Construa as funções a seguir:

a) Dado um produto, devolve seu preço.

b) Dado um elemento de QuantidadeVencimento, devolve a quantidade total de produtos.

c) Dado um produto, devolve o valor do estoque deste produto.

d) Dado um produto, baixa o preço em 30% se o produto for perecível e o estoque de produtos com vencimento no mês for maior que 30. O registro fica inalterado caso contrário.

e) Dado um produto, se ele for perecível, zera o estoque de produtos que vencem no mês.

f) Dados dois produtos, se os dois forem perecíveis, devolve o numero de itens que vencem neste mês de cada produto, caso contrário, devolve a mensagem "Erro: produto não perecível".

# Funções

## Seletores

- .preco
- .quant
- .nome
- .cod
- .prox
- .mes
- .seg

## Construtores

- produto
- quant
- venc-perecivel

## Testes de Tipo

- is-produto
- is-venc-perecivel
- is-quant

## Condicionais

- ask
- cases

## Operações

- +
- *
- and
- >

